

How I Built a Production-Grade Metric Logging System

for Apache Spark (PySpark) and Created a Custom UI Dashboard

A scalable, modular, and production-ready solution to collect, store, and visualize PySpark application metrics.



Built for PySpark
Seamless metric collection from Spark applications



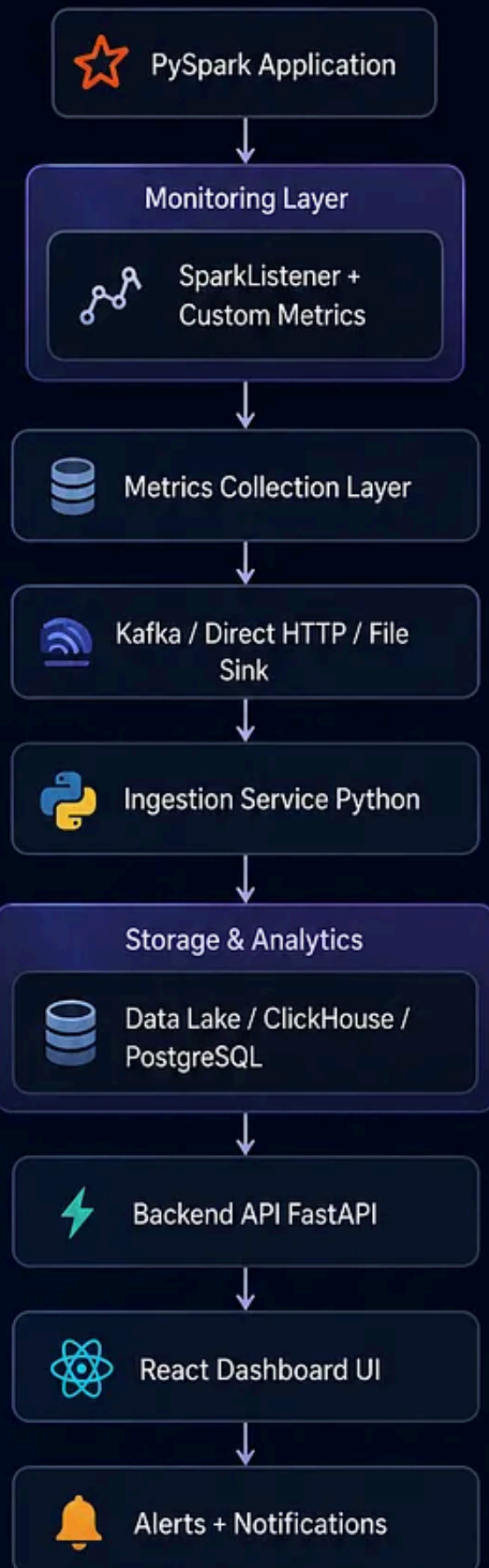
Reliable Storage
Persist metrics in ClickHouse / PostgreSQL for fast analytics



Custom Dashboard
Real-time insights with a beautiful React UI



Stay Informed
Get alerts & notifications when it matters



How I Built a Production-Grade Metric Logging System for Apache Spark (PySpark) and Created a Custom UI Dashboard



Anurag Sharma · 4 min read · May 25, 2026



1



1

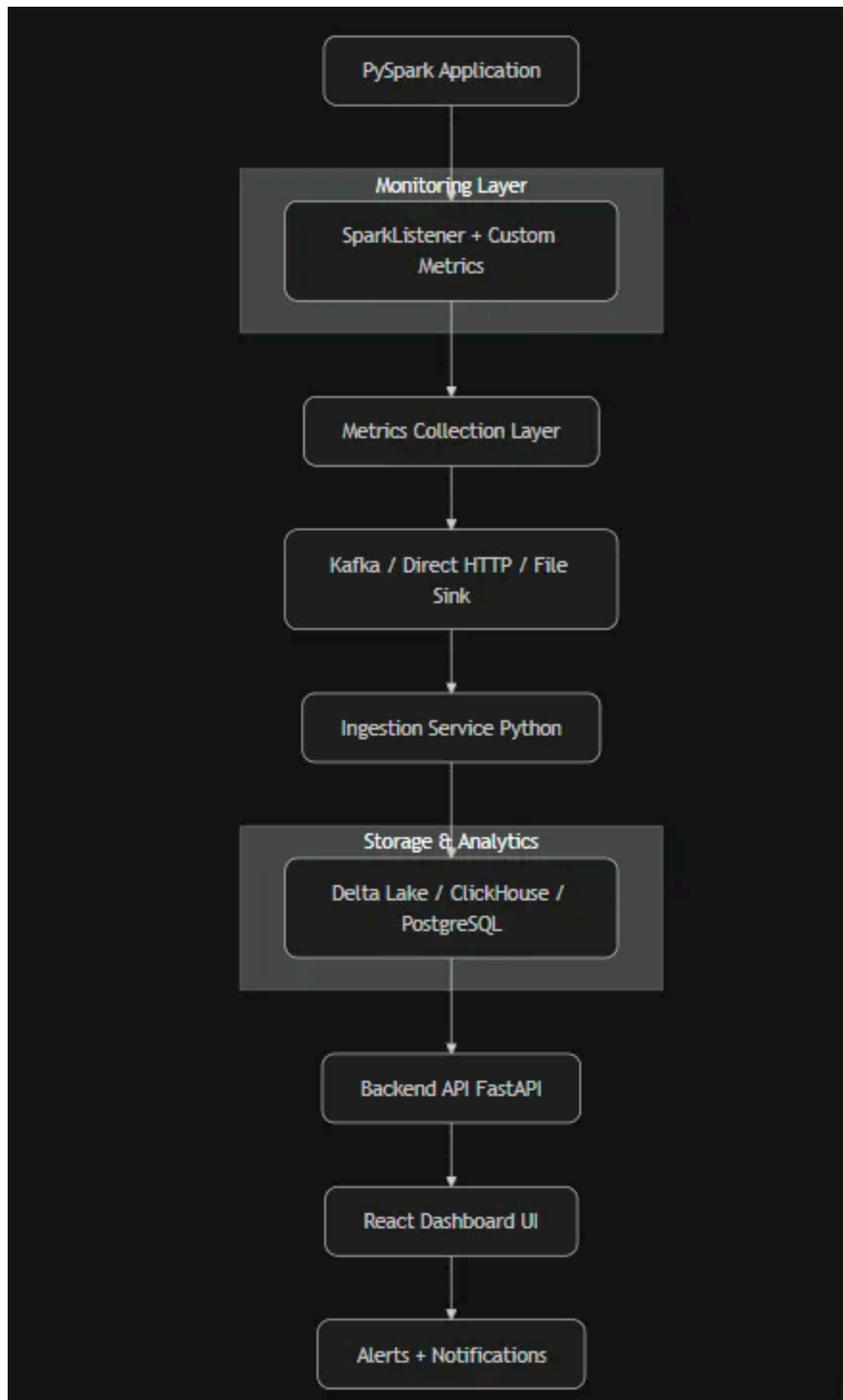


In large-scale data pipelines, the standard Spark UI is excellent for real-time debugging but falls short for historical analysis, custom business metrics, trend tracking, alerting, and team-wide visibility. I built a complete **metric logging and monitoring system** tailored for PySpark applications, with a modern web UI on top.

The Problem We Faced

- No persistent history of job/stage metrics across runs.
- Difficulty tracking custom business KPIs (e.g., records processed per customer segment, error rates by data type).
- Teams are wasting time switching between Spark UIs, logs, and monitoring tools.
- Need for performance regression detection and resource optimization.

High-Level Architecture



Core Flow:

1. Metrics are captured inside Spark (system + custom + business).

2. Sent to a central store.
3. A Python backend serves the data.
4. React UI provides rich visualizations.

Step 1: Metric Collection in PySpark

Option A: Using Accumulators for Simple Custom Business Metrics (Python)

```
from pyspark.sql import SparkSession
from pyspark.accumulators import AccumulatorParam
import json
import time

class MetricAccumulator(AccumulatorParam):
    def zero(self, initialValue):
        return initialValue

    def addInPlace(self, v1, v2):
        # Merge two metric dicts
        for k, v in v2.items():
            if k in v1:
                if isinstance(v, (int, float)):
                    v1[k] += v
                else:
                    v1[k] = v # overwrite for latest
            else:
                v1[k] = v
        return v1

spark = SparkSession.builder.appName("Metrics").getOrCreate()

# Register accumulator
metrics_acc = spark.sparkContext.accumulator({}, MetricAccumulator())

def record_metric(key: str, value):
    current = metrics_acc.value
    current[key] = current.get(key, 0) + value if isinstance(value, (int, float))
    metrics_acc.add({key: value})
```

Usage in transformations (use foreachPartition for efficiency):

```
def process_partition(iterator):
    local_metrics = {"records_processed": 0, "errors": 0}
    for row in iterator:
        try:
            # process row
            local_metrics["records_processed"] += 1
        except Exception:
            local_metrics["errors"] += 1
    # Send to accumulator (thread-safe)
    metrics_acc.add(local_metrics)
    yield ... # processed data

df.rdd.foreachPartition(process_partition)
```

Option B: Custom SparkListener (Scala)

SparkListener is implemented in Scala/Java. For PySpark, the best practice is to write a lightweight Scala listener JAR and attach it.



Example Scala Listener (compile to JAR):

```
// CustomMetricsListener.scala
package com.example.spark

import org.apache.spark.scheduler._
import org.apache.spark.sql.SparkSession
import java.time.Instant
```

```

class CustomMetricsListener extends SparkListener {
  override def onStageCompleted(stageCompleted: SparkListenerStageCompleted): Unit = {
    val info = stageCompleted.stageInfo
    val metrics = Map(
      "app_id" -> SparkSession.getActiveSession.map(_.sparkContext.applicationId),
      "stage_id" -> info.stageId,
      "num_tasks" -> info.numTasks,
      "stage_duration_ms" -> info.completionTime.getOrElse(0L),
      "input_bytes" -> info.taskMetrics.inputMetrics.bytesRead,
      "shuffle_write_bytes" -> info.taskMetrics.shuffleWriteMetrics.bytesWritten,
      "timestamp" -> Instant.now().toEpochMilli
    )
    // Push to Kafka, HTTP endpoint, or Delta Lake
    sendMetrics(metrics)
  }

  private def sendMetrics(metrics: Map[String, Any]): Unit = {
    // Example: HTTP post or Kafka producer
    println(s"Metrics: $metrics") // Replace with real sink
  }
}

```

Attach the listener in PySpark:

```

spark = (SparkSession.builder
  .appName("MyApp")
  .config("spark.extraListeners", "com.example.spark.CustomMetricsListener")
  .config("spark.driver.extraClassPath", "/path/to/your-listener.jar")
  .config("spark.executor.extraClassPath", "/path/to/your-listener.jar")
  .getOrCreate())

```

Step 2: Transport Layer

I used Kafka for high throughput:

```
# In the listener sink or a separate consumer
from kafka import KafkaProducer
producer = KafkaProducer(bootstrap_servers='kafka:9092') # your kakfa broker

def send_to_kafka(topic: str, metrics: dict):
    producer.send(topic, json.dumps(metrics).encode('utf-8'))
```

Alternative: Direct write to Delta Lake from driver.

Step 3: Ingestion & Storage Service (Python)

A lightweight FastAPI service consumes from Kafka and writes to Delta Lake.

```
# ingestion_service.py
from fastapi import FastAPI
from delta import *
import pyspark
from datetime import datetime

# Spark session for ingestion
spark = (SparkSession.builder
        .appName("ingestion_service")
        .getOrCreate())

app = FastAPI()

@app.post("/metrics")
async def receive_metrics(metrics: dict):
    df = spark.createDataFrame([metrics])
    df.write.format("delta").mode("append").partitionBy("date").save("/delta/spa
    return {"status": "ok"}
```

Storage Schema Example (Delta Lake):

- spark_apps: app-level summary
- spark_stages: detailed stage metrics
- business_metrics: custom KPIs with dimensions (team, domain, etc.)

Step 4: Building the UI Dashboard

Backend (FastAPI):

```
# api/main.py
from fastapi import FastAPI
from pyspark.sql import SparkSession

app = FastAPI(title="Spark Metrics API")

spark = SparkSession.builder.appName("Dashboard").getOrCreate()

@app.get("/apps")
def get_apps():
    df = spark.sql("SELECT * FROM delta.`/delta/spark_metrics`")
    return df.toPandas().to_dict(orient="records")

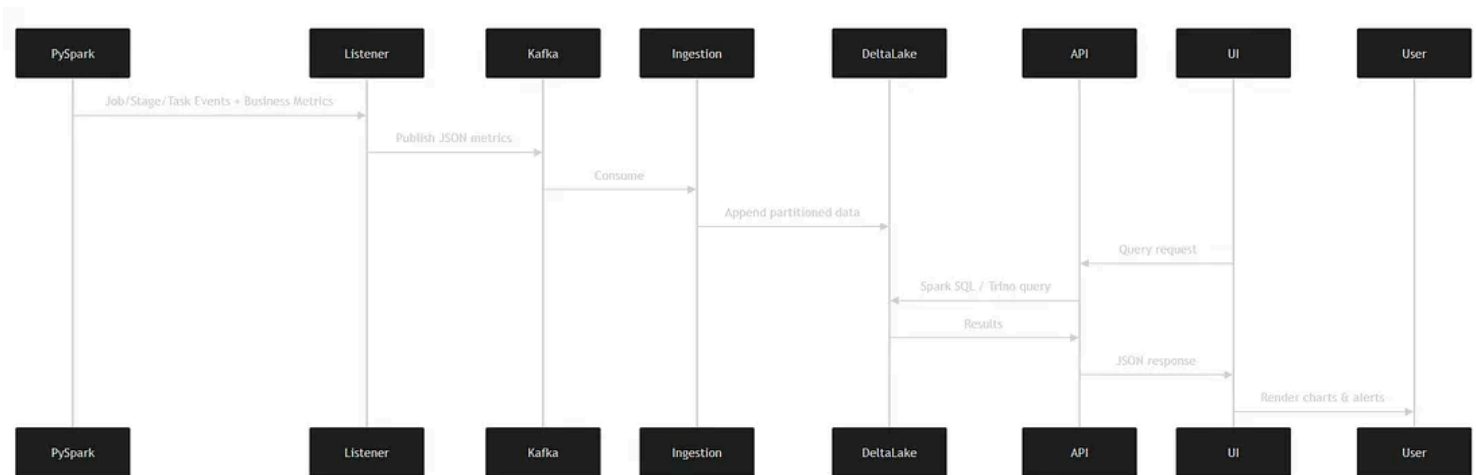
@app.get("/stage-metrics/{app_id}")
def get_stage_metrics(app_id: str):
    # Query with filters, aggregations, etc.
```

Frontend (React + TypeScript + Recharts/ECharts):

- Application list with success rate, average duration, and trends.
- Drill down into stages with Gantt-like timelines.
- Business metrics charts (line, bar, heatmaps for skew).
- Comparison between job runs.
- Alert rules configuration.

For quick wins, I also connected **Grafana** directly to the Delta Lake / ClickHouse.

Flow Diagram: End-to-End Metric Lifecycle



Key Challenges & How I Solved Them

1. **High Cardinality & Volume** — Aggregated metrics at ingestion + used time-based partitioning and downsampling in long-term tables.
2. **Listener Overhead** — Made sending asynchronous and batched.
3. **PySpark + Scala Bridge** — Kept listener minimal; all heavy logic in Python.
4. **Skew Detection** — Added custom logic in the listener to flag stages with high task duration variance.
5. **Schema Evolution** — Used Delta Lake for schema-on-read flexibility.

Results Achieved

- Debugging time reduced by 60–70%.

- Early detection of performance issues and data skew.
- Better resource allocation and cost savings.
- Single source of truth for data and platform teams.

Lessons Learned

- Start simple with accumulators → graduate to full SparkListener.
- Combine system metrics (shuffle, spill, GC) with business metrics.
- Persistence + historical comparison is more valuable than real-time alone.
- Grafana for 80% of needs; custom React UI for polished experience and workflows.

[Data Engineering](#)[Data Analysis](#)[Big Data](#)[Apache Spark](#)[Python Programming](#)

Written by Anurag Sharma

125 followers · 5 following

[Edit profile](#)

Data Engineering Specialist with 10+ exp. Passionate about optimizing pipelines, data lineage, and Spark performance and sharing insights to empower data pros!